

Warp feasibility analysis for a catering-company automation service

Executive summary

I evaluated whether Warp (Warp Terminal + Oz agent platform) can power a catering-company automation service *using Warp as the primary automation/orchestration layer*, based on the 33-item baseline extraction available now (the Google Drive requirements doc will supersede this once provided). Warp is not a “catering ops SaaS” out of the box; it is an agentic development environment with a programmable cloud-agent platform (Oz) that can be triggered by schedules, integrations, the CLI, or an HTTP API/SDK. ¹

My bottom line: **Warp can implement a substantial portion of the needed automation if you’re willing to treat your catering automation as an “automation codebase” (repo + Docker environment) and use Oz cloud agents as the execution runtime.** The strongest fit is: (1) cron-based weekly cadence automations, (2) Slack-based human-in-the-loop operations (“@Oz do X”), and (3) programmable triggers from your own systems via Oz API/SDK. ²

Key constraints that materially affect feasibility:

- **Warp cloud agents expect an “environment” with a Docker image + GitHub repos + setup commands**, which you’ll need to create even if the business is spreadsheet-native. ³
- **First-party business-app connectors (Google Workspace, Close CRM, ClickUp) are not the core focus of Warp integrations**; you’ll rely on custom code and/or MCP servers to reach spreadsheet/calendar surfaces. (Warp does support MCP broadly, including remote MCP endpoints over HTTP/SSE.) ⁴
- **Cloud agents do not currently support image attachments in the conversational UI**, which makes invoice/reimbursement OCR workflows more engineering-heavy (still doable via file attachments + OCR inside the environment, but not “just attach a photo”). ⁵
- Some limits are **explicitly not fully specified publicly** (e.g., cloud-agent concurrency details are “still being finalized”), so production sizing needs a quick pilot. ⁶

I recommend a **Warp-first reference architecture** where Oz cloud agents run your automation repo on a schedule and via Slack triggers, with optional webhook handlers calling Oz API for event-driven triggers. This approach aligns well with the extracted weekly rhythms, recurring cycles, and multi-surface coordination (calendar → staffing → prep → finance). ⁷

Assumptions and evaluation framework

I am making these assumptions (explicitly, since the Drive link is not yet provided):

- I used the **33-item baseline extraction** currently available as the “requirements list.” This is a proxy until the Google Drive doc is shared.

- I interpret **“using Warp alone”** as: using Warp/Oz as the orchestration + agent runtime (local agents + cloud agents + Oz CLI + Oz API/SDK + Warp Drive + MCP), while still integrating with your existing business systems (e.g., calendars/spreadsheets/CRM) via their public APIs or via MCP servers. Warp itself must remain the *primary* automation engine. ⁸
- I classify each requirement as:
- **Fully supported:** Warp provides the necessary primitives (triggers + runtime + secrets + integrations/API) and the rest is “implementation work in your automation repo,” without needing a separate automation platform.
- **Partially supported:** feasible, but blocked by a Warp limitation (e.g., image attachments in cloud agents), missing first-party connector, or requires a material workaround (custom service, third-party MCP server, etc.).
- **Not supported:** no practical path with Warp/Oz as the primary engine, given currently documented capabilities.

I also assume the business stack implied by the extraction includes: Google ⁹ Workspace surfaces (Sheets/Calendar), Close ¹⁰ (CRM), ClickUp ¹¹ (tasks), Slack ¹² (ops comms), plus a mixed Microsoft ¹³ / GoDaddy ¹⁴ environment, and social channels like YouTube ¹⁵, TikTok ¹⁶, Instagram ¹⁷, LinkedIn ¹⁸, and X ¹⁹.

Warp capability catalog relevant to business automation

Oz cloud agents, triggers, and programmable execution

Warp includes **Oz**, an orchestration platform for cloud agents. It is designed for running agent tasks from schedules, integrations, and other triggers, with shared observability of what ran and what it did. ²⁰

Core automation primitives (the ones that matter for a catering automation service):

- **Triggers:** Oz cloud agents can be triggered via scheduled agents (cron), CLI, API/SDK, and integrations. ²¹
- **Scheduled agents (cron):** Warp supports scheduled cloud agents that run automatically based on cron expressions; each run starts fresh, runs a fixed prompt/skill, consumes credits, and produces run history. ²²
- **HTTP API + official SDKs:** The Oz API lets you create and inspect cloud agent runs over HTTP from external systems (CI, cron, backend services). Warp provides official Python and TypeScript SDKs. ²³
- **CLI for automation:** The Oz CLI is explicitly positioned as a headless entry point usable in scripts and automated systems. ²⁴

Warp’s docs even describe webhook-driven automation as a common pattern: “register a webhook to your handler → handler calls the Oz API/SDK to start a task.” ²⁵

Environments and execution model

Cloud agents run “inside an environment,” and that environment defines **container image, repos, and setup steps** used when a trigger kicks off an agent task. ³

This is the key design implication for catering automation: **you will almost certainly create a dedicated GitHub repo that contains your automation code**, with a Docker image + setup commands so agents can run repeatable tasks (e.g., extract calendar events, update sheets, calculate forecasts, post Slack summaries). ²⁶

Execution hosting options:

- Default: Warp-hosted sandboxes.
- Enterprise: self-hosted execution (managed worker or unmanaged patterns), but the control plane and LLM inference remain Warp-managed per the documented split architecture. ²⁷

Integrations, connectors, and “webhooks”

Warp’s first-party integrations are strongly centered on dev workflows, but the trigger mechanics are general-purpose:

- **Slack integration:** Tagging @Oz in Slack starts an agent run; Warp posts updates back into the thread and shares a live session link. ²⁸
- **Team requirement for integrations:** Warp notes that cloud agents can be used individually via CLI/API, but integrations like Slack and Linear require team membership. ²⁹
- **Custom triggers:** Oz API/SDK can be called from your own systems, which is how you implement webhook-driven “business events” (new lead, event booked, invoice uploaded). ³⁰

MCP: extending Warp with external tools and data sources

Warp supports MCP (Model Context Protocol) as a plugin mechanism to expose tools/data to agents. For local agents, Warp supports remote MCP endpoints (Streamable HTTP/SSE) and CLI-launched MCP servers; for CLI/cloud agent runs you can pass MCP configs to `oz agent run` / `oz agent run-cloud` via `--mcp`. ³¹

This is one of the most practical routes to integrating spreadsheet-native operations with Warp **without writing every connector from scratch**, because you can adopt an MCP server for Google Drive/Docs/Sheets/Calendar (third-party) and bring it into your agent runtime. As an example of the broader MCP ecosystem, there are open-source MCP servers that claim Google Drive/Docs/Sheets/Calendar support. ³² (These are not Warp-official; I treat them as optional workarounds if you want MCP-native access to Google surfaces.)

Model capabilities and multimodality

Warp supports multiple LLM providers and models via “Model Choice,” including many OpenAI GPT-5 variants, multiple Anthropic Claude variants, and Google Gemini models. ³³

Multimodal constraints:

- **Local agents:** You can attach images (up to 5 per request and up to 20 across a conversation), and Warp states images are sent to the model provider and “immediately discarded” (not stored on Warp servers). ³⁴

- **Cloud agents:** Cloud agent conversations **do not currently support image attachments.** ⁵

For “non-image multimodal,” Warp supports attaching URLs as context for local agents (Warp scrapes the page content for context), but that is distinct from web search. ³⁵

Credits, pricing tiers, and practical limits

Warp’s current plan lineup includes Free, Build, Max, Business, and Enterprise, with plan-level differences in AI credits, model access, cloud conversation storage, and team/admin controls. ³⁶

Important non-financial constraints that affect automation:

- **Cloud agent concurrency/limits:** Warp explicitly notes that “concurrency limits and credit allocation details are still being finalized.” ⁶
- **Capacity signaling:** The Oz API’s run creation response includes an `at_capacity` field indicating whether the system is at capacity when a run is created (useful for robust orchestration). ³⁷
- **No public request-per-minute limit found in Warp’s Oz API docs** (in the materials I reviewed). I treat Oz API rate limits as **unspecified** and recommend designing with retry/backoff and queueing.

Key attributes table

Attribute to evaluate	What Warp provides (documented)	Plan notes / dependencies
Models	Multi-provider model choice across OpenAI GPT-5 variants, Anthropic Claude variants, Google Gemini models, and others; plus Auto modes that select models dynamically. ³⁸	Frontier model access varies by plan. ³⁶
Context window	Depends on selected model; Warp exposes a context window indicator and may auto-summarize when exceeded. ³⁹	Numeric per-model windows are not consistently listed in one place; token errors are described. ⁴⁰
Multimodal	Local agents support image attachments; cloud agents do not currently support image attachments. ⁵	If OCR/invoice workflows are core, expect additional engineering.
Orchestration	Oz cloud agents: run from triggers/schedules/integrations, with observable runs and transcripts. ⁴¹	Cloud agents included in Build; team integrations require team context. ⁴²
API	Oz API to create/monitor runs over HTTP; official Python + TypeScript SDKs. ²³	Requires API key; supports team vs personal keys. ⁴³
CLI	<code>oz</code> CLI to run cloud agents in scripts/automated systems; <code>warp-cli</code> deprecated. ²⁴	Works for individuals; integrations require teams. ⁴⁴

Attribute to evaluate	What Warp provides (documented)	Plan notes / dependencies
Scheduling	Scheduled agents via cron; manageable in Oz web app and API. ⁴⁵	Consumes credits per run. ⁴⁶
Webhooks	No “native inbound webhook endpoint” documented; recommended pattern is your webhook handler calls Oz API/SDK. ⁴⁷	Requires hosting a handler (serverless or service).
Connectors	First-party: Slack + Linear (and dev-centric flows). ⁴⁸ MCP: configurable servers over HTTP/SSE or CLI. ⁴	For Google Workspace/Close/ClickUp, likely custom code or third-party MCP.
Secrets	Cloud agent secrets store; encrypted at rest; injected at runtime; team-only secrets for scheduled agents. ⁴⁹	Encourages bot/service accounts for team secrets. ⁵⁰
Data retention / conversation storage	Cloud-synced conversations optional setting; cloud agent conversations always stored; storage limits vary by plan; free removes oldest when limit reached. ⁵¹	Enterprise admins can disable cloud conversation storage. ⁵²
Embeddings / indexing	Codebase Context uses an embeddings index; Warp says raw code is not stored, only embeddings retained. ⁵³	Intended for Git-tracked code; for business docs you’d likely build your own RAG.
Fine-tuning	No Warp-native fine-tuning documented; BYOK exists for interactive agents, but BYOK does not apply to cloud agents. ⁵⁴	If you need fine-tuning, it’s via external model providers (outside Warp).
ZDR / opt-out	ZDR agreements with contracted LLM providers are described; telemetry controls and plan-specific policies exist. ⁵⁵	Business/Enterprise: claims that no AI interaction or console data is collected (per privacy docs). ⁵⁶
Enterprise features	SSO/admin controls, self-hosted execution, BYOLLM via AWS Bedrock for interactive agents. ⁵⁷	BYOLLM currently applies to interactive agents; cloud agent support is “coming.” ⁵⁸

Security, privacy, compliance, and data lifecycle implications

Warp positions security around transparency, telemetry controls, secret redaction, encrypted storage, and ZDR terms with model providers. ⁵⁹

Zero Data Retention and model-provider handling

Warp states it has **Zero Data Retention (ZDR) agreements with contracted model providers** (including major providers) such that providers do not store or train on your data; the details vary by plan and configuration. ⁶⁰

Warp also publishes a ZDR note (PDF) describing ZDR as: no retention of AI inputs/outputs beyond immediate execution and no use of that content for product improvement when ZDR is selected (Business/Enterprise in the document). ⁶¹

Because this is plan/contract-dependent, I treat **the strongest “no-retention end-to-end” posture as a Business/Enterprise configuration question**, not a Free-plan baseline.

What is stored, where, and how long

Warp uses cloud features (Warp Drive, session sharing, cloud conversations, cloud agent runs) that store data in Warp-managed infrastructure if enabled/used. Warp’s privacy page states that if you choose to use cloud features like Warp Drive, saved items are sent to Warp servers and stored securely (encrypted at rest), and you can delete the data. ⁶²

Cloud conversation storage mechanics (important operationally):

- Cloud sync for local conversations is controlled by “Store AI conversations in the cloud.” ⁶³
- Even if you disable cloud sync, **cloud agent conversations are always stored in the cloud** (per Warp’s cloud conversation docs). ⁶³
- Free plans remove the oldest cloud conversations when you hit storage limits; storage limits vary by plan. ⁶⁴

Data retention in enterprise security overview includes: - Telemetry data (when collected) retained indefinitely. - User account deletion requests processed securely within 30 days (subject to legal/contractual obligations). ⁶⁵

Telemetry and opt-out posture

Warp’s docs describe: - Telemetry controls and real-time monitoring (Network Log). ⁶⁶

- On Free plan, telemetry must be enabled to use AI features; paid plans can opt out and still use Warp, including AI. ⁶⁷

- Business and Enterprise plan statements that no AI interaction/console data is collected (per the privacy doc). ⁵⁶

Secret handling and leakage risk

Warp provides secret redaction that uses regex patterns to redact sensitive information before sending data to Warp servers or LLM providers; users can add custom regexes. ⁶⁸

For cloud agents, Warp also documents a secrets store where secrets are encrypted at rest and injected at runtime, with differences in secret availability by trigger type (scheduled tasks get team-level secrets only). ⁶⁹

Practical implication for catering automation: if your automation code touches payroll/labor costs, customer PII, invoices, etc., **you should plan for rigorous secret redaction patterns plus strict secret scoping**, and consider disabling cloud conversation storage where required—but note cloud agent runs retain transcripts by design. ⁷⁰

Requirement mapping using the 33-item baseline extraction

The table below maps each extracted requirement/business-need area to Warp's documented capabilities. "Warp features" refers to Oz cloud agents + scheduled agents + Oz API/SDK/CLI + secrets + MCP + Slack triggers + Warp Drive objects as context.

Baseline requirement (1-33)	Warp features to use	Status	Notes / limitations	Recommended next steps
Overall business shape is structured around recurring workflows & documents	Scheduled agents; skills; Slack triggers; Oz API for event-driven runs; run audit logs	☐ Fully supported	Strong fit because Oz is built around repeatable automation units (runs + prompts/skills + schedules). ⁷¹	Create "automation repo" + environment; encode workflows as skills and scheduled agents; standardize outputs (Sheets + Slack).
Sales and lead management	Oz runs + CRM API calls; Slack command surface for sales reps	⚠ Partially supported	No first-party Close/CRM connector; you'll implement via API calls (code) or MCP server.	Build Close integration module; create runbooks/skills: "create lead," "update stage," "weekly closes."
Lead source attribution	Scheduled reporting agents + Sheets update + Slack summary	⚠ Partially supported	Requires defining a canonical source taxonomy + pulling data from CRM and/or lead tracker sheets.	Implement source normalization; scheduled "source mix & conversion" report.
Close CRM as core system	Cloud agents hitting Close API; secrets store for tokens	⚠ Partially supported	Works well if you treat Close as a normal external API; no Warp-native CRM UI.	Add Close API client; map Close fields to event pipeline and forecast models.
Lead tracker for cash forecasting	Scheduled agents; Oz API for on-demand forecasts	☐ Fully supported	Oz is excellent for recurring computation and producing artifacts; forecasting logic is your code.	Build a "cash forecast skill" that reads pipeline + calendar.

Baseline requirement (1–33)	Warp features to use	Status	Notes / limitations	Recommended next steps
Weekly labor reporting	Scheduled weekly agent; secrets; exports to Sheets	☐ Fully supported	Requires a canonical labor source (Sheets/ time-tracking/payroll export).	Implement weekly labor ETL + variance reporting.
Scheduling is major operational function	Scheduled planning runs; Slack “staffing draft” command; constraint solver in code	⚠ Partially supported	Warp doesn’t provide a scheduling UI; the schedule artifact is likely Sheets/Calendar updates and Slack messages.	Implement event → staffing recommendation; push schedule blocks into Calendar/Sheets.
Calendar is operational truth	Google Calendar API integration; scheduled event-ingestion agents	⚠ Partially supported	No first-party Google Calendar integration documented; doable via API or MCP.	Implement “calendar ingest” module; normalize to internal event schema.
Jobs/events require detailed operational fields	Skills that generate an “event brief” artifact; store in Drive/ Sheets	☐ Fully supported	You can generate consistent briefs and checklists if you maintain an event schema.	Define “Event Brief JSON” and auto-populate from Calendar + BEO links.
Visual readability matters	Generate color-coded Sheets + standardized Slack blocks	⚠ Partially supported	Warp doesn’t replace an ops dashboard; it can generate artifacts in systems that <i>are</i> visual (Sheets).	Standardize “Ops Sheet” formatting rules + automated template population.
Job complexity varies; implicit complexity model	Build scoring model; scheduled tuning; use for staffing/ procurement	☐ Fully supported	Works well as an internal scoring service + weekly recalibration.	Implement a complexity scoring function; store per-event labels and suggested staffing ratios.

Baseline requirement (1–33)	Warp features to use	Status	Notes / limitations	Recommended next steps
Reimbursements / expense handling	API-triggered run on receipt upload; scheduled reconciliation	⚠ Partially supported	Cloud agent image limitation affects “snap a receipt” UX. ⁷²	Use file attachments + OCR in environment (or external OCR); build reimbursement approval workflow via Slack.
Weekly financial visibility (in/out)	Scheduled weekly cash run; run artifacts; optional alerts	☐ Fully supported	Purely computational + reporting; needs bank/AR/AP data sources.	Implement “expected inflows/ outflows” aggregation from pipeline + bills.
Shared food shopping / prep list	Event → menu → ingredient rollups; scheduled list generation	☐ Fully supported	Excellent fit for scheduled planning outputs; integrates well with Sheets.	Create ingredient database + recipe scaling; generate weekly rollup and per-event prep list.
Role-specific task sheets	Generate role-specific views in Sheets; Slack pings; skills per role	⚠ Partially supported	Warp can automate generation, but role UX is in Sheets/Slack.	Define role-based task schemas; scheduled daily “manager sheet refresh.”
Venue relationships strategically important	Relationship database + reminder schedules	⚠ Partially supported	Needs a CRM-like surface or a structured sheet + reminders.	Schedule “venue touchpoints” agent; maintain venue owner assignments and reminders.
Event planners as relationship surface	Same as venue relationships	⚠ Partially supported	Same constraint: requires relationship data structure.	Extend relationship schema to planners; include referral attribution.

Baseline requirement (1–33)	Warp features to use	Status	Notes / limitations	Recommended next steps
Venue ownership assigned to reps	Scheduled audits; Slack nudges; owner field enforcement	☐ Fully supported	Simple governance automation + reporting.	Implement ownership rules + exception report for unowned venues.
Preferred vendors + location-aware recommending	Vendor directory + geo model; generate proposals in sales workflow	⚠ Partially supported	Requires vendor DB and location logic; recommendations can be generated textually.	Build vendor directory; integrate into proposal/tasting workflows.
Tastings are structured pipeline stage	Scheduled tasting calendar → follow-up tasks; Slack assistant	⚠ Partially supported	Requires integration with CRM + calendar + post-tasting data capture.	Implement tasting event templates; auto-create follow-ups and update Close stage.
Commission tracking exists	Scheduled commission report generation	☐ Fully supported	Straightforward compute + report once formulas are defined.	Implement commission rules engine; generate monthly and per-pay-period reports.
Google Sheets everywhere	Use Sheets as primary data store + artifact sink	⚠ Partially supported	Warp supports the agent runtime; Google access needs API/MCP.	Build/choose a Google Workspace connector strategy (API client or MCP).
Google is main environment	Same as above	⚠ Partially supported	Strong alignment if you treat Google as your “UI + DB.”	Consolidate operational truth into a small number of canonical Sheets.

Baseline requirement (1–33)	Warp features to use	Status	Notes / limitations	Recommended next steps
Tool stack fragmentation (Google + Microsoft)	Normalize ingestion; multi-source connectors; unified reporting	⚠ Partially supported	Requires connector work (Microsoft/ Outlook + Google).	Implement a “unified calendar ingestion” layer + dedupe rules.
Org has multiple functional roles	Team-based triggers; Slack interface; permissioned secrets	⚠ Partially supported	Warp teams/admin are dev-centric; role-based business permissions require your own policies.	Define role to Slack-channel mapping; restrict which skills can be invoked by whom.
Wants to scale through systems + people	Scheduled automation + SOP generation + alerts	☐ Fully supported	Oz is built to codify repeatable workflows; humans get escalations via Slack.	Encode SOPs as skills; add exception-driven alerting.
Management layer exists (manager sheets)	Scheduled daily summaries; “blocked items” notifications	☐ Fully supported	Warp management view helps observe runs; your manager UI is Slack/Sheets. ⁷³	Build a daily “manager digest” scheduled agent + weekly planning run.
Weekly rhythms dominate	Scheduled agents + recurring runbooks	☐ Fully supported	Scheduled agents are natively cron-based. ⁷⁴	Encode the weekly operating cycle as 3–6 scheduled agents (Mon planning, Thu confirmations, Sun recap).
Recurring operational cycles (lead→event→reconcile)	Multi-trigger flows; API + schedule + Slack	☐ Fully supported	Oz is designed for trigger→run→artifact patterns. ⁷⁵	Model the business as a state machine; create agents to advance state and produce artifacts.

Baseline requirement (1–33)	Warp features to use	Status	Notes / limitations	Recommended next steps
Creating order out of complexity (visual grouping, summaries)	Artifact generation + standardized templates	☐ Fully supported	Warp excels at templated outputs; “visual” aspect lives in Sheets.	Define canonical dashboards and let scheduled agents populate them.
Social media interest	Scheduled content pipeline; prompt templates; optional MCP to scheduling tools	⚠ Partially supported	Warp is not a social scheduler; but can generate content and push to APIs if integrated.	Build a “content calendar” and “repurpose long→short” agents; connect to posting tools if needed.
Candidate for templating across similar businesses	Skill library + templates + reusable schema	☐ Fully supported	Warp Drive + skills are reusable; automation repo can become a template. ⁷⁶	Turn workflows into an “ops kit” repo: schemas, skills, dashboards, and deployment scripts.
Company is manually overextended (needs automation)	Automated scheduled + event-driven runs; Slack front-end	☐ Fully supported	Oz directly targets “delegate repeatable work to agents with auditability.” ⁷⁷	Start with workflows that reduce weekly ops overhead: calendar ingest, staffing draft, shopping list rollup, weekly cash view.

Missed automation opportunities and prioritization

These opportunities are commonly high-leverage in catering ops and align with the extracted needs. I list complexity and priority in practical engineering terms (relative, not absolute).

Opportunity	What it automates	Warp/Oz implementation sketch	Complexity	Priority
Event complexity scoring → staffing suggestion	Converts “implicit complexity model” into consistent staffing and prep decisions	Scheduled agent computes per-event score from standardized fields; publishes staffing draft to Sheets + Slack	Med	High
Menu personalization + quote generation	Create proposals + add-ons based on venue/planner preferences and constraints	Slack-triggered agent (“draft proposal for lead X”), pulls CRM + preferred vendors + historical menus, outputs PDF/Doc	High	High
Inventory + procurement forecasting	Weekly rollups of ingredients, rentals, disposables	Scheduled weekly agent aggregates menus → ingredient quantities → purchasing plan; alerts on shortages	High	High
OCR for reimbursements/ invoices	Turn receipts into structured reimbursements + accounting entries	Cloud agent fetches receipt files, runs OCR in environment, posts reimbursement approval request to Slack	High	High
Scheduling optimization	Constraint-based schedule across venues, travel, staff availability	Build solver in repo (e.g., MILP/heuristics); scheduled planning run proposes schedule alternatives	High	Med-High
Real-time “kitchen readiness” monitoring	Live checklists, critical-path alerts	Event-driven runs + Slack reminders + “missing items” detection; optional IoT is outside Warp	High	Med
Venue/planner relationship cadence automation	Standardized touchpoints, reminders, and logging	Scheduled reminders; Slack interactive prompts; updates CRM/sheets	Med	High
Commission automation + dispute audit	Trusted commission reports with traceability to deals	Monthly scheduled run computes commissions; produces audit artifact + Slack summary	Med	Med

Opportunity	What it automates	Warp/Oz implementation sketch	Complexity	Priority
Social content factory (from events)	Post-event content, captions, clips, platform-specific variants	Scheduled “content batch” based on event photos/ logs; <i>note cloud agents lack image attachments as conversation input</i>	High	Med
Ops anomaly detection	Catch “something’s off” early (labor spike, missing BEO, late payments)	Scheduled daily/weekly monitor runs + Slack alerts; uses thresholds and simple models	Med	High

Recommended production architecture and rollout

Production architecture

The architecture below treats Warp/Oz as the agent-runtime and orchestration control plane, with your catering automation logic living in a dedicated repo and running in a reproducible environment. This matches Warp’s environment model (Docker image + repos + setup commands) and trigger model (schedule + integrations + API). ⁷⁸

```

flowchart LR
    subgraph Trigger_Surfaces["Trigger surfaces"]
        Cron["Scheduled Agents (cron)"]
        SlackTrig["Slack @Oz mention"]
        APITrig["Your systems (webhooks / backend)"]
        Manual["Manual run (Oz web app / oz CLI)"]
    end

    subgraph WarpOz["Warp / Oz platform"]
        Orchestrator["Oz orchestration + run tracking"]
        Secrets["Secrets store (team + personal)"]
        MCP["MCP servers (optional)"]
        Audit["Run artifacts + transcripts + status"]
    end

    subgraph Execution["Execution"]
        Runner["Runner (Warp-hosted or self-hosted)"]
        Env["Environment: Docker image + automation repo + setup commands"]
        Code["Automation code: ETL + rules + forecasting + reporting"]
    end

    subgraph BusinessSystems["Business systems"]
        Workspace["Google Workspace (Sheets/Calendar/Drive)"]
        CRM["Close CRM"]
    end

```

```

    Tasks["ClickUp (optional)"]
    Comms["Slack channels + DMs"]
end

Cron --> Orchestrator
SlackTrig --> Orchestrator
APITrig --> Orchestrator
Manual --> Orchestrator

Orchestrator --> Runner
Secrets --> Env
MCP --> Env
Runner --> Env
Env --> Code
Code <--> BusinessSystems

Code --> Audit
Orchestrator --> Audit
Audit --> Comms

```

Rollout roadmap

```

timeline
    title Implementation roadmap (high-level)
    Phase 1 : Create automation repo + Oz environment + secrets store + minimal
runbooks
    Phase 2 : Calendar ingest → event briefs → weekly shopping/prep rollups
(scheduled agent)
    Phase 3 : Close CRM + lead attribution + weekly close + commission reporting
    Phase 4 : Forecasting + optimization (labor, inventory, schedule) +
reimbursement OCR pipeline
    Phase 5 : Social content operations (content calendar + repurposing) + venue/
planner relationship cadence automation

```

Practical “start here” build sequence (lowest regret)

I recommend starting with workflows that match the extraction’s weekly cadence and reduce the most manual overhead first:

- 1) **Weekly planning pack:** scheduled agent that ingests next 7–14 days of events and produces (a) event briefs, (b) staffing draft, (c) prep/shopping rollups, (d) cash in/out expectations. This aligns directly with scheduled agents and Oz API. ⁷⁹
- 2) **Slack as the front-end:** let sales/ops trigger “@Oz” workflows (“summarize this event,” “create shopping list,” “draft follow-up”) and deliver artifacts back into the thread. ²⁸
- 3) **Google Workspace integration strategy:** decide whether you will integrate via direct APIs in your repo or adopt an MCP server approach; Warp supports MCP over HTTP/SSE and via CLI configs. ⁴

4) **Governance:** implement strict secrets hygiene (team secrets for scheduled runs) and define which workflows are allowed per role; Warp supports profiles/permissions for agent autonomy/tool use. 80

If/when you share the Google Drive requirements doc, I will replace the baseline mapping with a requirement-by-requirement mapping from your actual user stories and acceptance criteria.

1 8 20 41 Overview | Agents

https://docs.warp.dev/agent-platform?utm_source=chatgpt.com

2 7 18 22 45 46 71 74 79 Scheduled Agents

https://docs.warp.dev/agent-platform/cloud-agents/triggers/scheduled-agents?utm_source=chatgpt.com

3 15 78 Environments | Agents

https://docs.warp.dev/agent-platform/cloud-agents/environments?utm_source=chatgpt.com

4 31 Model Context Protocol (MCP) | Agents

https://docs.warp.dev/agent-platform/capabilities/mcp?utm_source=chatgpt.com

5 34 Images as Context | Agents

https://docs.warp.dev/agent-platform/local-agents/agent-context/images-as-context?utm_source=chatgpt.com

6 9 42 49 72 77 FAQs | Agents

https://docs.warp.dev/agent-platform/cloud-agents/faqs?utm_source=chatgpt.com

10 27 58 Self-Hosting | Agents

https://docs.warp.dev/agent-platform/cloud-agents/self-hosting?utm_source=chatgpt.com

11 73 Managing Cloud Agents

https://docs.warp.dev/agent-platform/cloud-agents/managing-cloud-agents?utm_source=chatgpt.com

12 17 51 52 63 64 70 Cloud-synced Conversations | Agents | Warp

<https://docs.warp.dev/agent-platform/local-agents/cloud-conversations>

13 14 36 57 Warp Pricing: Terminal, Agents & Cloud Platform

https://www.warp.dev/pricing?utm_source=chatgpt.com

16 19 56 59 66 67 Privacy | Support & Community | Warp

<https://docs.warp.dev/support-and-community/privacy-and-security/privacy>

21 Triggers | Agents

https://docs.warp.dev/agent-platform/cloud-agents/triggers?utm_source=chatgpt.com

23 30 Oz API & SDK | Reference | Warp

<https://docs.warp.dev/reference/api-and-sdk/api-and-sdk>

24 Oz CLI | Reference

https://docs.warp.dev/reference/cli/cli?utm_source=chatgpt.com

25 47 Deployment Patterns | Agents

https://docs.warp.dev/agent-platform/cloud-agents/deployment-patterns?utm_source=chatgpt.com

26 Oz Web App | Agents

https://docs.warp.dev/agent-platform/cloud-agents/oz-web-app?utm_source=chatgpt.com

28 48 **Slack | Agents**

https://docs.warp.dev/agent-platform/cloud-agents/integrations/slack?utm_source=chatgpt.com

29 44 **Access, Billing, and Identity | Agents**

https://docs.warp.dev/agent-platform/cloud-agents/team-access-billing-and-identity?utm_source=chatgpt.com

32 **Google Drive MCP Server**

https://github.com/piotr-agier/google-drive-mcp?utm_source=chatgpt.com

33 38 **Model Choice | Agents**

https://docs.warp.dev/agent-platform/capabilities/model-choice?utm_source=chatgpt.com

35 **URLs as Context | Agents**

https://docs.warp.dev/agent-platform/local-agents/agent-context/urls-as-context?utm_source=chatgpt.com

37 **Agent | Reference | Warp**

<https://docs.warp.dev/reference/api-and-sdk/agent>

39 **Interacting with Agents**

https://docs.warp.dev/agent-platform/local-agents/interacting-with-agents?utm_source=chatgpt.com

40 **FAQs | Agents**

https://docs.warp.dev/agent-platform/getting-started/faqs?utm_source=chatgpt.com

43 **API Keys | Reference**

https://docs.warp.dev/reference/cli/api-keys?utm_source=chatgpt.com

50 69 **Secrets | Agents**

https://docs.warp.dev/agent-platform/cloud-agents/secrets?utm_source=chatgpt.com

53 **Codebase Context | Agents**

https://docs.warp.dev/agent-platform/capabilities/codebase-context?utm_source=chatgpt.com

54 **Bring Your Own API Key | Support & Community**

https://docs.warp.dev/support-and-community/plans-and-billing/bring-your-own-api-key?utm_source=chatgpt.com

55 60 65 **Security overview | Enterprise**

https://docs.warp.dev/enterprise/security-and-compliance/security-overview?utm_source=chatgpt.com

61 **Warp Zero Data Retention**

https://leading-bell-3e1c02e64d.media.strapiapp.com/Warp_Zero_Data_Retention_Note_0828_fdd115f847.pdf?utm_source=chatgpt.com

62 **Your privacy while using Warp**

https://www.warp.dev/privacy?utm_source=chatgpt.com

68 **Secret Redaction | Support & Community**

https://docs.warp.dev/support-and-community/privacy-and-security/secret-redaction?utm_source=chatgpt.com

75 **Oz Platform | Agents**

https://docs.warp.dev/agent-platform/cloud-agents/platform?utm_source=chatgpt.com

76 **Skills | Agents**

https://docs.warp.dev/agent-platform/capabilities/skills?utm_source=chatgpt.com

80 **Profiles & Permissions | Agents**

https://docs.warp.dev/agent-platform/capabilities/agent-profiles-permissions?utm_source=chatgpt.com